Lecture 8. Factors, Dates and Times

forcats, lubridate: creating, components, arithmetic, time zones

Contents

8.1 Factors **2**

8.1.1 Creating factors . . . 2
8.1.2 The General Social Survey . . . 2
8.1.3 Modifying factor order . . . 2
8.1.4 Modifying factor levels . . . 3

8.2 Dates and times **4**

8.2.1 Creating dates and date-times . . . 4
8.2.2 Date-time components . . . 5
8.2.3 Time spans . . . 6
8.2.4 Time zones . . . 7

8.1 Factors

Motivation. Categorical variables need two things that plain strings cannot give: a fixed set of allowed values, so that typos are caught, and an order that is not alphabetical, so that months sort as months and plots show categories in a meaningful sequence. Factors provide both, and forcats makes them easy to manipulate.

8.1.1 Creating factors

Recording months as strings has two problems: only twelve values are possible but nothing protects against typos, and they sort alphabetically.

```
x1 <- c("Dec", "Apr", "Jan", "Mar")
x2 <- c("Dec", "Apr", "Jam", "Mar")
sort(x1)
```

```
## [1] "Apr" "Dec" "Jan" "Mar"
```

```
month_levels <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
y1 <- factor(x1, levels = month_levels)
sort(y1)
```

```
## [1] Jan Mar Apr Dec
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

```
y2 <- factor(x2, levels = month_levels)
y2
```

```
## [1] Dec  Apr  <NA> Mar
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

A factor is created from a vector and its valid **levels**; values not in the levels silently become `NA` (`readr::parse_factor()` warns instead). Without `levels`, the levels are the sorted unique values; `factor(x1, levels = unique(x1))` keeps the order of first appearance. `levels(y1)` lists the levels.

8.1.2 The General Social Survey

`forcats::gss_cat` is a sample from the General Social Survey, a long-running US survey, with many factor columns (marital status, race, income, party, religion). Factor levels can be seen with `count()` or a bar chart; `scale_x_discrete(drop = FALSE)` shows levels that have no observations.

```
gss_cat %>% count(race)
```

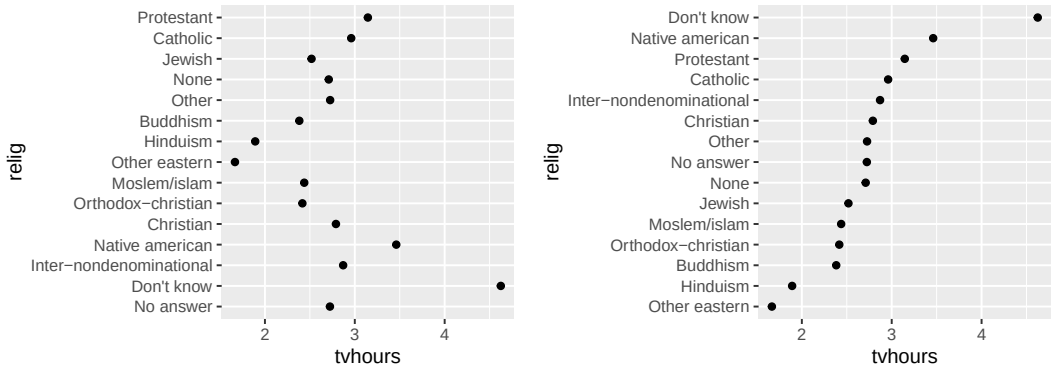
```
## # A tibble: 3 x 2
##   race      n
##   <fct> <int>
## 1 Other  1959
## 2 Black  3129
## 3 White 16395
```

The two most common operations on factors are changing the *order* of the levels and changing their *values*.

8.1.3 Modifying factor order

Average daily TV hours by religion is hard to read with the default level order. `fct_reorder(f, x)` reorders the levels of `f` by the median of `x` (or another function given as `.fun`):

```
relig_summary <- gss_cat %>%
  group_by(relig) %>%
  summarise(age = mean(age, na.rm = TRUE),
            tvhours = mean(tvhours, na.rm = TRUE), n = n())
ggplot(relig_summary, aes(tvhours, relig)) + geom_point()
relig_summary %>%
  mutate(relig = fct_reorder(relig, tvhours)) %>%
  ggplot(aes(tvhours, relig)) + geom_point()
```



Reordering inside `mutate()` rather than inside `aes()` keeps the plotting code simple. Do not reorder factors whose levels already have a natural order: for reported income, `fct_reorder(rincome, age)` would scramble the income brackets. Instead, `fct_relevel(rincome, "Not applicable")` only moves the named level(s) to the front.

Other reordering tools: `fct_reorder2(f, x, y)` orders by the `y` values at the largest `x`, so that line colours match the legend; `fct_infreq(f)` orders by frequency (useful for bar charts, combined with `fct_rev()` to reverse).

8.1.4 Modifying factor levels

`fct_recode()` renames levels (new name on the left) and leaves unmentioned levels unchanged; assigning several old levels to one new name combines them:

```
gss_cat %>%
  mutate(partyid = fct_recode(partyid,
    "Republican, strong"    = "Strong republican",
    "Republican, weak"      = "Not str republican",
    "Independent, near rep" = "Ind,near rep",
    "Independent, near dem" = "Ind,near dem",
    "Democrat, weak"        = "Not str democrat",
    "Democrat, strong"      = "Strong democrat",
    "Other"                 = "No answer",
    "Other"                 = "Don't know",
    "Other"                 = "Other party"
  )) %>%
  count(partyid)
```

```
## # A tibble: 8 x 2
##   partyid                 n
##   <fct>               <int>
## 1 Other                 548
## 2 Republican, strong   2314
## 3 Republican, weak     3032
## 4 Independent, near rep 1791
## 5 Independent          4119
## # i 3 more rows
```

Combining levels this way should be done with care, since groups that really differ get merged. To collapse many levels, `fct_collapse()` takes a vector of old levels for each new one:

```
gss_cat %>%
  mutate(partyid = fct_collapse(partyid,
    other = c("No answer", "Don't know", "Other party"),
    rep = c("Strong republican", "Not str republican"),
    ind = c("Ind,near rep", "Independent", "Ind,near dem"),
    dem = c("Not str democrat", "Strong democrat"))) %>%
  count(partyid)
```

```
## # A tibble: 4 x 2
##   partyid     n
##   <fct>   <int>
## 1 other     548
## 2 rep      5346
## 3 ind      8409
## 4 dem      7180
```

`fct_lump(f, n = 10)` keeps the `n` most frequent levels and lumps the rest into "Other", useful for plots and tables of variables with many rare levels.

8.2 Dates and times

Motivation. Time looks simple but is not: years are not all the same length, days in daylight-saving changes have 23 or 25 hours, and the same instant has different clock readings in different places. lubridate separates these issues into clear concepts so that time arithmetic gives the answer you expect.

`lubridate` (not loaded by `library(tidyverse)` in older versions) handles three types: a **date** (`<date>`), a **time** within a day (`<time>`, from the `hms` package), and a **date-time** (`<dttm>`, called POSIXct in base R), a date plus a time uniquely identifying an instant. Always use the simplest type that works: a date if you do not need the time, since date-times bring time-zone complications. `today()` and `now()` return the current date and date-time.

8.2.1 Creating dates and date-times

From strings. Name the order of year (`y`), month (`m`) and day (`d`) and lubridate parses the rest; add `_h`, `_hm`, `_hms` for times. They also accept unquoted numbers.

```
ymd("2017-01-31")
```

```
## [1] "2017-01-31"
```

```
mdy("January 31st, 2017")
```

```
## [1] "2017-01-31"
```

```
dmy("31-Jan-2017")
```

```
## [1] "2017-01-31"
```

```
ymd_hms("2017-01-31 20:11:59")
```

```
## [1] "2017-01-31 20:11:59 UTC"
```

```
mdy_hm("01/31/2017 08:01")
```

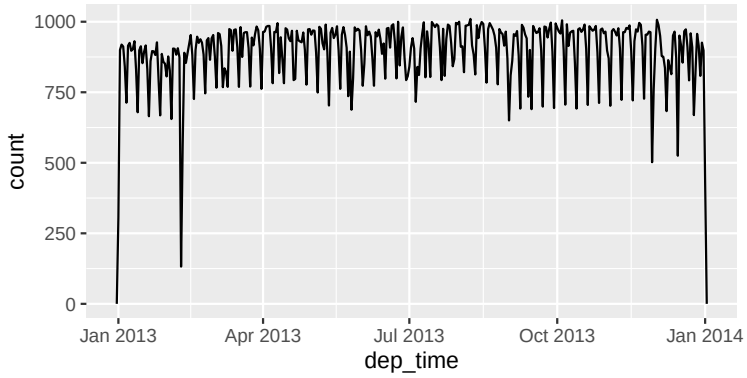
```
## [1] "2017-01-31 08:01:00 UTC"
```

From components. `make_date()` and `make_datetime()` build values from columns. In `flights`, times are stored in `HHMM` form, so we write a small helper that splits them with modular arithmetic:

```
make_datetime_100 <- function(year, month, day, time) {
  make_datetime(year, month, day, time %/% 100, time %% 100)
}
flights_dt <- flights %>%
  filter(!is.na(dep_time), !is.na(arr_time)) %>%
  mutate(
    dep_time = make_datetime_100(year, month, day, dep_time),
    arr_time = make_datetime_100(year, month, day, arr_time),
    sched_dep_time = make_datetime_100(year, month, day, sched_dep_time),
    sched_arr_time = make_datetime_100(year, month, day, sched_arr_time)
  ) %>%
  select(origin, dest, ends_with("delay"), ends_with("time"))
```

In a numeric context a date-time counts seconds (and a date counts days), so a histogram binwidth of 86400 is one day:

```
flights_dt %>%
  ggplot(aes(dep_time)) +
  geom_freqpoly(binwidth = 86400)
```

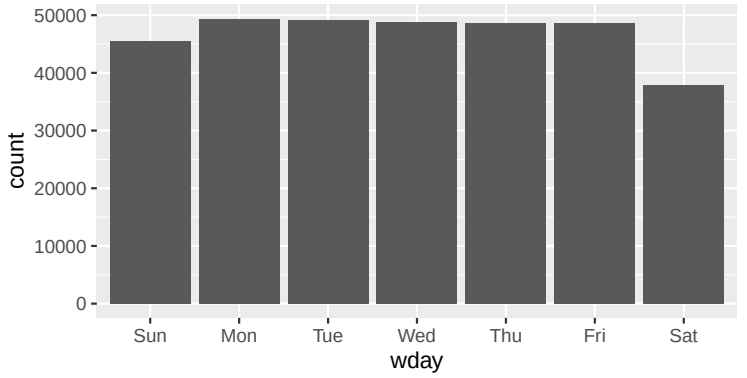


From other types. `as_datetime()` and `as_date()` switch between the two; given numbers they interpret them as an offset from the "Unix epoch" 1970-01-01.

8.2.2 Date-time components

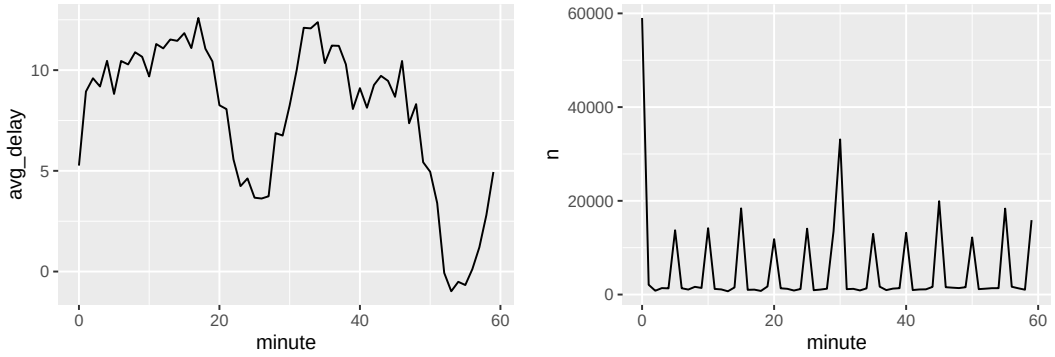
Accessors extract components: `year()`, `month()`, `mday()` (day of month), `yday()` (day of year), `wday()` (day of week), `hour()`, `minute()`, `second()`. With `label = TRUE`, `month()` and `wday()` return abbreviated names. Fewer flights leave at weekends:

```
flights_dt %>%
  mutate(wday = wday(dep_time, label = TRUE)) %>%
  ggplot(aes(x = wday)) + geom_bar()
```



The average delay by minute of *actual* departure shows that flights leaving at minutes 20–30 and 50–60 have much lower delays. By minute of *scheduled* departure there is no such pattern; the pattern comes from people preferring "nice" departure times, so early departures cluster there [Supplement].

```
flights_dt %>%
  mutate(minute = minute(dep_time)) %>%
  group_by(minute) %>%
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE), n = n()) %>%
  ggplot(aes(minute, avg_delay)) + geom_line()
sched_dep <- flights_dt %>%
  mutate(minute = minute(sched_dep_time)) %>%
  group_by(minute) %>%
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE), n = n())
ggplot(sched_dep, aes(minute, n)) + geom_line()
```



The right panel shows the strong preference for scheduled departures on round minutes. Look for this kind of human rounding whenever data are produced by people.

Rounding to a unit: `floor_date()`, `round_date()`, `ceiling_date()`, e.g. the number of flights per week is `count(week = floor_date(dep_time, "week"))`. **Setting** components: assign to an accessor, `year(datetime) <- 2020`, or use `update()` to change several at once; values that overflow roll over (setting `mday = 30` in February moves into March). `update(dep_time, yday = 1)` puts all flights on the same day, which shows the distribution of departures over the hours of the day.

```
datetime <- ymd_hms("2016-07-08 12:34:56")
update(datetime, year = 2020, month = 2, mday = 2, hour = 2)
```

```
## [1] "2020-02-02 02:34:56 UTC"
```

8.2.3 Time spans

Motivation. "Plus one day" can mean exactly 86 400 seconds or "the same clock time tomorrow", and these differ across a daylight-saving change. lubridate keeps them apart as durations and periods.

Subtracting two dates gives a `difftime`, whose unit varies. lubridate offers three classes:

- **Durations** are an exact number of seconds: `as.duration(today() - ymd(19791014))`; constructors `dseconds()`, `dminutes()`, `dhours()`, `ddays()`, `dweeks()`, `dyears()`. Because they count physical seconds, adding `ddays(1)` across a daylight-saving change gives an unexpected clock time.
- **Periods** are human units: `seconds()`, `days()`, `months()`, `years()`. Adding `days(1)` keeps the clock time, and `years(1)` respects leap years. Periods fix flights that arrive after midnight: their arrival time was built on the departure date, so add `days(1)` when `arr_time < dep_time`.

- **Intervals** are a duration with a start point, `start %--% end`. Dividing an interval by a period gives the exact number of, e.g., months it contains, which a duration cannot (months have different lengths).

```
one_pm <- ymd_hms("2016-03-12 13:00:00", tz = "America/New_York")
one_pm + ddays(1)     # daylight saving started overnight: 14:00
```

```
## [1] "2016-03-13 14:00:00 EDT"
```

```
one_pm + days(1)      # same clock time: 13:00
```

```
## [1] "2016-03-13 13:00:00 EDT"
```

```
next_year <- today() + years(1)
(today() %--% next_year) / days(1)
```

```
## [1] 365
```

Rule of thumb: use a duration if you care about physical time, a period if you need human time, and an interval if you need to measure a span in human units.

8.2.4 Time zones

Time zones are named by "continent/city" (IANA names such as `"Asia/Seoul"`, `"America/New_York"`); `Sys.timezone()` gives yours and `OlsonNames()` lists all. The time zone only affects printing: the same instant printed in three zones is still the same instant, and the differences between them are 0. lubridate uses UTC (Coordinated Universal Time) by default, which has no daylight saving. To change the time zone:

- `with_tz(x, "Asia/Seoul")` keeps the instant and changes the display;
- `force_tz(x, "Asia/Seoul")` keeps the clock reading and changes the instant, for values that were labelled with the wrong time zone.

```
x <- ymd_hms("2024-06-01 12:00:00", tz = "UTC")
with_tz(x, "Asia/Seoul")
```

```
## [1] "2024-06-01 21:00:00 KST"
```

```
force_tz(x, "Asia/Seoul") - x
```

```
## Time difference of -9 hours
```